Interfaces funcionais

Algoritmia e Estruturas de Dados (AED)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEETC, ADEETC

Slides baseados nos slides da professora Manuela Sousa

ISEL

Resumo

- Interface funcionais base: `Predicate`, `Function`, `Consumer`, e `Supplier`
- Interfaces funcionais base com dois argumentos
- Especializações das interface funcional `Function`
- Interfaces funcionais para tipos primitivos

O que é?

- Conjunto de interfaces adequadas para serem usadas com expressões *lambda*, ou seja, interfaces que definem apenas um único método abstrato
  - De notar que, qualquer interface que defina um único método abstrato, pode ser considerada uma *interface funcional*: pode incluir vários métodos estáticos e implementações *default* (*default methods)*
  - A anotação `@FunctionalInterface` permite que o compilador verifique se a interface é, de facto, uma interface funcional

- Incluídas na biblioteca standard do Java a partir da versão 8
  - Interfaces funcionais, a par de expressões *lambda* e referências para métodos impulsionam a programação funcional, com escrita de código mais simples e mais legível

- As *interfaces funcionais* estão definidas no *package* `java.util.function`

Resumo

- Interface funcionais base: `Predicate`, `Function`, `Consumer`, e `Supplier`
- Interfaces funcionais base com dois argumentos
- Especializações das interface funcional `Function`
- Interfaces funcionais para tipos primitivos

Interfaces funcionais base

- *Predicate*: permite verificar se o valor do parâmetro tem determinada característica

```
@FunctionalInterface public interface Predicate<T> {
        boolean test(T t); // Retorna true caso t verifique a característica.
}
```

- *Function*: permite retornar o resultado da transformação do parâmetro de entrada

```
@FunctionalInterface public interface Function<T, R> {
        R apply(T t); // Retorna o resultado da transformação de t.
}
```

- *Consumer*: permite realizar ações sobre o parâmetro de entrada

```
@FunctionalInterface public interface Consumer<T> {
        void accept(T t); // Executa a ação sobre t.
}
```

- *Supplier*: permite retornar um resultado

```
@FunctionalInterface public interface Supplier<R> {
        R get(); // Produz um valor do tipo R.
}
```

Exemplo com `Predicate`

```
public class PredicateDemo {
   public static int predicateDemo(List<Integer> list, Predicate<Integer> filter) {
      int count = 0;
      for(Integer v: list)
         if (filter.test(v))  count += 1;
      return count;
   }
   public static void main(String[] args) {
      List<Integer> l = Arrays.asList(1, 100, 230, 23, 48, 23, 63, 65, 23, 50);  int cnt;
      Predicate<Integer> size = _ -> true;
      Predicate<Integer> even = i -> i%2 == 0;
      cnt = predicateDemo(l, size); System.out.println("Size: " + cnt);
      cnt = predicateDemo(l, even); System.out.println("Evens: " + cnt);
} }
```

Exemplo com `Predicate` with `Generic`

```
public class PredicateDemo {
   public static <T> int predicateDemo(List<T> list, Predicate<T> filter) {
      int count = 0;
      for(T v: list)
         if (filter.test(v))  count += 1;
      return count;
   }
   public static void main(String[] args) {
      List<Integer> il = Arrays.asList(1, 100, 230, 23, 48, 23, 63, 65, 23, 50);  int cnt;
      cnt = predicateDemo(il, i -> i%2 == 1); System.out.println("Odds: " + cnt);
      List<String> sl = Arrays.asList("água", "terra", "ar", "fogo");
      cnt = predicateDemo(sl, s -> s.length() > 4); System.out.println("Above4Letters: " + cnt);
      cnt = predicateDemo(sl, s -> s.contains("a")); System.out.println("ContainsALetter: " + cnt);
}  }
```

Exemplo com `Function`

```
public class FunctionDemo {
   public static <T,R> List<R> functionDemo(List<T> list, Function<T,R> map) {
      ArrayList<R> r = new ArrayList<>();
      for(T v: list)
         r.add(map.apply(v));
      return r;
   }
   public static void main(String[] args) {
      List<Integer> il = Arrays.asList(1, 100, 230, 23, 48, 23, 63, 65, 23, 50);
      Function<Integer, Double> factor2p5 = i -> i*2.5;
      List<Double> dl = functionDemo(il, factor2p5); // print elems
      List<String> sl = Arrays.asList("água", "terra", "ar", "fogo");
      Function<String, String> reverse = s -> new StringBuilder(s).reverse().toString();
      List<String> rsl = functionDemo(sl, reverse); // print elems
} }
```

Exemplo com `Consumer`

```
class Struct {
   Struct(int x, int y) { this.a = x; this.b = y; }
   int a, b, r;
   public String toString() { return "r = " + r; }
}
public class ConsumerDemo {
   public static <T> void ConsumerDemo(List<T> list, Consumer<T> consumer) {
      for(T v: list)   consumer.accept(v);
   }

   public static void main(String[] args) {
      List<Integer> il = Arrays.asList(1, 100, 230, 23, 48, 23, 63, 65, 23, 50);
      ConsumerDemo(il, System.out::println);
      List<Struct> xl = Arrays.asList(new Struct(10, 20), new Struct(15,20), new Struct(15, 4));
      ConsumerDemo(xl, x -> x.r = x.a + x.b);  System.out.println(xl);
      ConsumerDemo(xl, x -> x.r = x.a - x.b);   System.out.println(xl);
}  }
```

Exemplo com `Supplier`

```
public class SupplierDemo {
   public static <R extends List<S>,T,S> R supplierDemo(List<T> list, Supplier<R> supplier, Function<T,S> map) {
      R r = supplier.get();
      for(T v: list) r.add(map.apply(v));
      return r;
   }
   public static void main(String[] args) {
      List<Integer> il = Arrays.asList(1, 100, 230, 23, 48, 23, 63, 65, 23, 50);
      Supplier<List<Double>> col1 = ArrayList::new; Function<Integer, Double> factor2p5 = i -> i*2.5;
      List<Double> dl = supplierDemo(il, col1, factor2p5); System.out.println(dl);
      List<String> sl = Arrays.asList("água", "terra", "ar", "fogo");
      Supplier<List<String>> col2 = LinkedList::new;
      Function<String, String> reverse = s -> new StringBuilder(s).reverse().toString();
      List<String> rsl = supplierDemo(sl, col2, reverse); System.out.println(rsl);
} }
```

Resumo

- Interface funcionais base: `Predicate`, `Function`, `Consumer`, e `Supplier`
- Interfaces funcionais base com dois argumentos
- Especializações das interface funcional `Function`
- Interfaces funcionais para tipos primitivos

Funções com dois argumentos

- As interfaces funcionais `BiPredicate`, `BiFunction` e `BiConsumer` têm a mesma funcionalidade das interfaces `Predicate`, `Function` e `Consumer` com a diferença que têm dois parâmetros de entrada

Interface	Descrição
`BiPredicate<T, U>`	Representa um predicado que testa dois argumentos **`- boolean test(T t, U u)`**
`BiFunction<T, U, R>`	Representa uma função que realiza operações sobre os dois argumentos e produz um resultado **`- R apply(T t, U u)`**
`BiConsumer<T, U>`	Representa uma função que realiza ações sobre dois argumentos **`- void accept(T t, U u)`**

Resumo

- Interface funcionais base: `Predicate`, `Function`, `Consumer`, e `Supplier`
- Interfaces funcionais base com dois argumentos
- Especializações das interface funcional `Function`
- Interfaces funcionais para tipos primitivos

Especializações de `Function`

- As interfaces `UnaryOperator` e `BinaryOperator` estendem `Function` e `BiFunction`, respetivamente, e operam sobre valores do mesmo tipo

```
public interface UnaryOperator<T> extends Function<T, T> {}
public interface BinaryOperator<T> extends BiFunction<T, T, T> {}
```

Interface	Descrição
`UnaryOperator<T>`	Representa uma função que realiza operações sobre um único argumento e produz um resultado do mesmo tipo **`- T apply(T t)`**
`BiOperator<T>`	Representa uma função que realiza operações sobre dois argumentos do mesmo tipo e produz um resultado, igualmente, do mesmo tipo **`- T apply(T t1, T t2)`**

Resumo

- Interface funcionais base: `Predicate`, `Function`, `Consumer`, e `Supplier`
- Interfaces funcionais base com dois argumentos
- Especializações das interface funcional `Function`
- Interfaces funcionais para tipos primitivos

Interfaces funcionais para tipos primitivos - `double`

Interface	Descrição
`DoublePredicate`	Representa predicado que testa valor do tipo `double` - **`boolean test(double d)`**
`DoubleFunction<R>`	Representa operação sobre `double` e produz resultado do tipo R **- `R apply(double d)`**
`DoubleConsumer`	Representa ação sobre `double` **- `void accept(double d)`**
`DoubleSupplier`	Representa fornecedor de valores do tipo `double` **- `double getAsDouble()`**
`DoubleUnaryOperator`	Representa operação sobre um `double` e produz resultado do tipo `double` **- `double applyAsDouble(double d)`**
`DoubleBinaryOperator`	Representa operação sobre dois `doubles` e produz resultado do tipo `double` **- `double applyAsDouble(double d1, double d2)`**
`DoubleToIntFunction`	Representa operação sobre um `double` e produz resultado do tipo `int` **- `int applyAsInt(double d)`**
`DoubleToLongFunction`	Representa operação sobre um `double` e produz resultado do tipo `long` **- `long applyAsLong(double d)`**

Interfaces funcionais para tipos primitivos - `int`

Interface	Descrição
`IntPredicate`	Representa predicado que testa valor do tipo `int` - **`boolean test(int i)`**
`IntFunction<R>`	Representa operação sobre `int` e produz resultado do tipo R **- `R apply(int i)`**
`IntConsumer`	Representa ação sobre `int` **- `void accept(int i)`**
`IntSupplier`	Representa fornecedor de valores do tipo `int` **- `int getAsInt()`**
`IntUnaryOperator`	Representa operação sobre um `int` e produz resultado do tipo `int` **- `int applyAsInt(int i)`**
`IntBinaryOperator`	Representa operação sobre dois `ints` e produz resultado do tipo `int` **- `int applyAsInt(int i1, int i2)`**
`IntToDoubleFunction`	Representa operação sobre um `int` e produz resultado do tipo `double` **- `double applyAsDouble(int i)`**
`IntToLongFunction`	Representa operação sobre um `int` e produz resultado do tipo `long` **- `long applyAsLong(int i)`**

Interfaces funcionais para tipos primitivos - `long`

Interface	Descrição
`LongPredicate`	Representa predicado que testa valor do tipo `long` - **`boolean test(long l)`**
`LongFunction<R>`	Representa operação sobre `long` e produz resultado do tipo R **- `R apply(long l)`**
`LongConsumer`	Representa ação sobre `long` **- `void accept(long l)`**
`LongSupplier`	Representa fornecedor de valores do tipo `long` **- `long getAsLong()`**
`LongUnaryOperator`	Representa operação sobre um `long` e produz resultado do tipo `long` **- `long applyAsLong(long l)`**
`LongBinaryOperator`	Representa operação sobre dois `longs` e produz resultado do tipo `long` **- `long applyAsLong(long l1, long l2)`**
`LongToDoubleFunction`	Representa operação sobre um `long` e produz resultado do tipo `double` **- `double applyAsDouble(long l)`**
`LongToIntFunction`	Representa operação sobre um `long` e produz resultado do tipo `int` **- `int applyAsInt(long l)`**

Interfaces funcionais com retornos de tipos primitivos

Interface	Descrição
`ToDoubleFunction<T>`	Representa operação sobre valor do tipo T e produz resultado do tipo `double` - **`double applyAsDouble(T t)`**
`ToDoubleBiFunction<T, U>`	Representa operação sobre dois valores do tipo T e U e produz resultado do tipo `double` **- `double applyAsDouble(T t, U u)`**
`ToIntFunction<T>`	Representa operação sobre valor do tipo T e produz resultado do tipo `int` **- `int applyAsInt(T t)`**
`ToIntBiFunction<T, U>`	Representa operação sobre dois valores do tipo T e U e produz resultado do tipo `int` **- `int applyAsInt(T t, U u)`**
`ToLongFunction<T>`	Representa operação sobre valor do tipo T e produz resultado do tipo `long` **- `long applyAsLong(T t)`**
`ToLongBiFunction<T, U>`	Representa operação sobre dois valores do tipo T e U e produz resultado do tipo `long` **- `long applyAsLong(T t, U u)`**